

ACMEMobility

Progetto di Architetture Software a Microservizi, Università di Bologna, 2025–2026

Daniele D’Ugo
daniele.dugo@studio.unibo.it
Mat.: 0001186581

Daniele D’Ugo
daniele.dugo@studio.unibo.it
Mat.: 0001186581

Daniele D’Ugo
daniele.dugo@studio.unibo.it
Mat.: 0001186581

I. INTRODUZIONE

Il servizio di noleggio ACMEMobility è stato implementato come un’architettura a microservizi, tramite *docker compose*, cercando di rispettare tutti i requisiti del progetto.

II. AVVIO E UTILIZZO

L’intera architettura, comprendente tutti i servizi, con il codice nella cartella *src/* può essere avviata e gestita tramite i comandi di *docker compose*; per comodità, sono presenti due script *bash* (per sistemi Linux). *start.sh* lancia l’infrastruttura:

```
1 # dalla cartella di base del repository
2 ./src/start.sh
```

down.sh interrompe i servizi:

```
1 ./src/down.sh
```

Prima, però, è necessario scaricare la mappa (con dati di OpenStreetMap) dell’Italia nord-orientale, disponibile al link su geofabrik: <https://download.geofabrik.de/europe/italy/nord-est-latest.osm.pbf>.

Il file dovrà essere piazzato in *service-graphhopper/data/*, con nome del file *geofabrik_italy-nord-est-260517.osm.pbf*.

Da Linux, si può eseguire direttamente, per entrambi i passaggi, il comando:

```
1 # dalla cartella di base del repository
2 wget -O service-graphhopper/data/geofabrik_italy-nord-est-260517.osm.pbf \
3 https://download.geofabrik.de/europe/italy/nord-est-latest.osm.pbf
```

Il flusso delle attività parte sempre dall’utente, che in uno scenario reale sarebbe un’entità fisica e potrebbe interagire con ACMEMobility tramite un’applicazione per smartphone. Dunque, abbiamo realizzato uno script in *python* (*src/user/user.py*) che simula tutte le possibili azioni che possa compiere, in base agli argomenti utilizzati. Seguono degli esempi per ogni utilizzo. Usage:

```
1 # dalla cartella di base del repository
2 python src/user/user.py
3 # Usage: python user.py <action> <user_id> <vehicle_id> [station_id]
4 # Actions: scan, reserve, cancel, scan_reserved, park, lock, lock_assistance, steal
```

Scansione veicolo (con id *V-01*) per uso immediato, da parte dell’utente con id *USER-001*:

```
1 python src/user/user.py scan USER-001 V-01
```

Prenotazione veicolo per dopo:

```
1 python src/user/user.py reserve USER-001 V-01
```

Cancellazione della prenotazione precedente:

```
1 python src/user/user.py cancel USER-001 V-01
```

Scansione del veicolo precedentemente prenotato:

```
1 python src/user/user.py scan_reserved USER-001 V-01
```

Parcheggio del veicolo (simulazione dell'inserimento del veicolo nel dock della stazione con id *STATION-BOLOGNA-01*):

```
1 python src/user/user.py park USER-001 V-01 STATION-BOLOGNA-01
```

Richiesta di fine corsa e, dunque, blocco del veicolo precedentemente parcheggiato:

```
1 python src/user/user.py lock USER-001 V-01 STATION-BOLOGNA-01
```

Richiesta di assistenza a fine corsa, in caso di problemi con il blocco:

```
1 python src/user/user.py lock_assistance USER-001 V-01
```

Simulazione furto di un veicolo:

```
1 python src/user/user.py steal USER-001 V-01
```

Una volta avviata l'infrastruttura, è possibile accedere alla visualizzazione di Camunda al link <http://localhost:8080/camunda/app/cockpit/>, con username *demo* e password *demo*.

III. WORKFLOW E ARTEFATTI

Lo sviluppo del progetto è partito con la modellazione di una coreografia in BPMN con scopo documentativo. Usandola come schema è stato così possibile suddividere, tra i membri del gruppo, i compiti di realizzare ulteriori modelli o i singoli servizi.

La coreografia è stata poi gradualmente raffinata, fino a diventare la attuale coreografia a scopo documentativo (file *docs/diagrams/diagram-collaboration-full.bpmn*). Essa raffigura infatti tutti i partecipanti e i loro task, per rendere l'idea di quali feature debba implementare ogni servizio.

Per Camunda è stato modellato un altro diagramma di collaborazione eseguibile (file *src/camunda-resources/diagram-collaboration-camunda.bpmn*), con la principale differenza che non implementa nel dettaglio tutti i partecipanti, ma solo il servizio principale di ACMEMobility, che era richiesto che fosse gestito tramite BPMS.

Successivamente, sono stati realizzati un diagramma di documentazione della SOA in UML, con profilo TinySOA (file *docs/diagrams/diagram-SOA.uml* e *docs/diagrams/diagram-SOA-UML.uml*), e una coreografia con sintassi formale, necessaria per le proiezioni della coreografia sui ruoli e per l'analisi di *connectedness*.

I servizi implementati, invece, sono:

- ACME (*src/service-acme-worker/*): il worker che implementa i service task del BPMS per il core di ACMEMobility, scritto in *python* con l'apposito pacchetto per Camunda;
- bank (*src/service-bank/*): la banca, in *jolie*;
- Fleet Management (*src/service-fleet-management/*): implementato come un router, in *Nginx*, così che appaia esso stesso come un'architettura a microservizi;
 - Fleet Management Battery (*src/service-fleet-battery/*): microservizio per gestire la batteria dei veicoli, in *python* con *Flask*;
 - Fleet Management Tracking (*src/service-fleet-tracking/*): microservizio per gestire posizione e informazioni aggiuntive dei veicoli, in *python* con *Flask*;
- grapphopper (*src/service-graphhopper/*): routing service per fornire dei percorsi reali ai veicoli, date posizione di partenza e destinazione, nonché per visualizzarli in tempo reale; configurato con il container *docker* già esistente;
- station (*src/service-stations/*): implementazione di una singola stazione, in *python* con *Flask*;
- vehicle (*src/service-vehicle/*): implementazione di un singolo veicolo, in *python* con *Flask*.

Tutti i servizi sono gestiti tramite un'architettura con *docker-compose*, che si occupa anche di istanziare molteplici veicoli e stazioni, con la stessa immagine e implementazione. Il file è scritto come template in

src/docker-compose.template.yml; uno script *bash* (*src/bash-scripts/compile-dockercompose.sh*, già invocato da *start.sh*) lo lavora, per esempio sostituendo delle variabili con quelle in *src/.env*, generando quello finale (*src/docker-compose.generated.yml*). In questo modo, i veicoli e le stazioni istanziati sono ben riconoscibili poiché generati con identificativi definiti tramite variabili di ambiente.

Per la coreografia in Camunda, come già accennato, tutti i service task sono implementati tramite un worker in python. Solo delle operazioni banali, principalmente le condizioni di scelta per i gateway esclusivi, sono implementate come espressioni *inline*, con sintassi *JUEL* (*Java Unified Expression Language*), utilizzando delle variabili globali che vengono configurate dallo stesso worker. L'implementazione esterna, infatti, fornisce grandi vantaggi nello sviluppo, in termini di gestibilità, mantenibilità e leggibilità del codice, possibilità di debugging e gestione coerente e ordinata dei file.

Inoltre, tutti i servizi usano *API REST* e comunicano tramite *http*, ad eccezione della banca, che usa *SOAP* come richiedo.

In *docs/implementation/* sono presenti le documentazioni sia per le API dei servizi (incluso ACMEMobility, a cui lo script *user.py* invia le richieste), sia per le variabili globali di Camunda (*bpms-variables.md*).

IV. IMPLEMENTAZIONE

In questa sezione presentiamo il modo in cui vengono eseguiti i processi (e quindi la coreografia), insieme a commenti e motivazioni per le scelte di design.

A. La coreografia

1) *Prospettiva user*: Come già accennato, il primo processo a partire è quello dell'**utente**, che va poi ad avviare a catena gli altri, con dei *message start event*. Nella **fase 1**, la **prenotazione**, l'utente parte con una scelta: scansionare il codice QR di un veicolo subito oppure prenotarne uno; nel secondo caso, può in seguito scansionare il veicolo per far partire la corsa, oppure cancellare la prenotazione e quindi terminare il suo processo. In entrambi i casi, dopo essere giunto con successo alla scansione, inizia la **fase 2**, il **tragitto**, estremamente semplice dal punto di vista dell'utente, meno dal punto di vista di ACMEMobility. Infine, nella **fase 3**, la **conclusione**, l'utente deve interrompere la corsa e attendere la notifica di conferma da parte dell'applicazione. L'interruzione della corsa è in realtà ulteriormente articolata: bisogna prima inserire il veicolo in un terminale della stazione (chiamato anche "dock") e, successivamente, richiedere il blocco del veicolo ad ACMEMobility, tramite l'applicazione. In caso di problemi nel parcheggiare, si possono ripetere più volte le due operazioni di parcheggio e blocco finché non si abbia successo, oppure contattare l'assistenza (interrompendo il processo della coreografia): in questo modo, un addetto verrà inviato a controllare personalmente il veicolo, mentre per l'utente ACMEMobility procederà con l'addebito, in modo che possa andarsene e non debba rimanere ad aspettare. Qui, per gestire il fatto che il veicolo venga lasciato incustodito, assumiamo che, nel mondo reale, esso abbia qualche sistema di bloccaggio automatico della guida, o che magari funzioni solo quando un utente correttamente autorizzato (attraverso la **fase 1**) sia nelle vicinanze (il che potrebbe essere confermato dalla vicinanza del suo telefono, per esempio via *Bluetooth*).

Nel parcheggio, mentre il blocco del veicolo passa attraverso ACMEMobility (attraverso un *message intermediate throw event*), il precedente inserimento è modellato direttamente come un messaggio verso la stazione: nel mondo reale, infatti, immaginiamo che veicolo e terminale possano essere configurati con sensori per comunicazione a breve raggio (per esempio, per riconoscere un tag *RFID* o *NFC* sul veicolo), grazie ai quali la stazione può identificare se ci sia un veicolo inserito, prima di bloccarlo. D'altra parte, ACME, all'arrivo della notifica di parcheggio, procede, in ordine, a chiedere alla stazione il bloccaggio, effettuare l'addebito di fine corsa e confermare l'operazione all'utente con una notifica. La separazione in inserimento e bloccaggio è stata fatta per rendere più solido il design di questa parte: in questo modo, l'applicazione dell'utente deve solo comunicare gli identificativo di stazione e veicolo, e la stazione saprà automaticamente quale terminale bloccare; la comunicazione a breve raggio, inoltre, permette al veicolo e

stazione di scambiare eventuali informazioni di conferma.

2) *Prospettiva ACMEMobility*: Dal punto di vista di **ACMEMobility**, la coreografia appena descritta è più complessa e articolata.

Fase 1 (prenotazione) — un processo di ACME può partire solo da un messaggio, ricevuto dall'utente, di noleggio immediato o prenotazione: i due possibili processi si uniscono subito (con un gateway), ma l'informazione sul tipo di noleggio rimane memorizzata, in Camunda, come variabile globale (maggiori dettagli in seguito).

Subito dopo, si effettua il blocco della cauzione, richiedendolo alla banca. Per di più, solamente in caso di prenotazione, ACME effettua alcune operazioni interne, come aggiornare i database e salvarne un timestamp, e si mette in attesa di uno di tre possibili eventi: inizio della corsa (con scansione del veicolo da parte dell'utente), cancellazione della prenotazione o scadenza del tempo (30 minuti). In caso di annullamento della corsa (per richiesta o per scadenza del tempo), dunque, sblocca la cauzione o la converte in addebito in base al tempo trascorso, sempre tramite la banca.

Se e quando il processo arriva alla **fase 2 (tragitto)**, ACME si occupa di tenere monitorato il veicolo. Prima di tutto, chiede al servizio di *FleetManagement* di avviarne il tracciamento e, solo in seguito, alla stazione opportuna di sbloccarlo, notificando infine l'utente: così, si evita di far partire una corsa con un veicolo non raggiungibile. Durante lo spostamento, *FleetManagement* viene periodicamente interrogato (*polling*) riguardo allo stato del veicolo, finché non arrivi la richiesta di terminazione.

Nella **conclusione (fase 3)**, dopo aver bloccato il veicolo nel suo dock (tramite la stazione), ACME richiede (un'ultima volta) il suo livello della batteria, ne termina il tracking e usa l'informazione per determinare l'importo dovuto; infine, tramite la banca, effettua per primo l'addebito e per secondo lo sblocco della cauzione — tenendosela in caso di problemi di pagamento. Se, al momento del parcheggio, l'utente ha richiesto l'assistenza, si passa ugualmente per i task di calcolo e addebito, prima di gestire l'intervento manuale.

Andando a specificare alcune note aggiuntive, si assume che ACMEMobility conosca già credenziali, dati personali e dati bancari dell'utente — necessari per richiedere gli addebiti —, che verranno raccolti dal servizio in contesti fuori dalla modellazione richiesta dal progetto (fasi di registrazione e gestione dell'account), probabilmente appoggiandosi a un servizio esterno di gestione delle carte di credito per motivi di sicurezza. Per minimizzare lo scambio di informazioni riservate sulla rete, la banca ritorna un token alla prima chiamata, che può essere riusato in seguito per identificare una specifica sessione dell'utente, con la sua cauzione (eventualmente convertibile in addebito).

Oltre ad alcuni appena visti, la coreografia tiene conto anche di altri possibili casi di errore: se avvenuti prima di qualsiasi addebito (come al momento del blocco della cauzione), si invia solo un rifiuto di noleggio o prenotazione all'utente; se avvenuti dopo (come per avvio del tracking o sblocco del veicolo prima della corsa), si richiede lo sblocco della cauzione, prima di inviare una notifica. Per altri errori in fasi più critiche, si richiede l'intervento di un umano ma, allo stesso tempo, si manda sempre avanti, in qualche modo, il processo, con l'obiettivo di non lasciare mai l'utente fermo e costretto ad aspettare — d'altra parte, la coreografia ammette attese “asincrone”, come un pagamento ritardato di qualche ora, cioè che non lo bloccano fisicamente. Ciò avviene in particolare per gli errori in fase di conclusione e pagamento (richiesta del livello della batteria oppure addebito o sblocco della cauzione), in cui si invia una notifica di errore alternativa a quella di “pagamento avvenuto con successo”, così che il processo *user* possa terminare. In caso di impossibilità di interrompere il tracking, così come per la richiesta di assistenza in fase di parcheggio, essendo un errore che non inficia in alcun modo il pagamento, si procede con calcoli e addebito e contemporaneamente si richiede l'intervento di un umano, ma senza inviare messaggi di errore.

Non vengono mostrati invece eventuali problemi con il veicolo selezionato dall'utente (quando già prenotato, in uso, con batteria scarica o in manutenzione): assumiamo che, nel lato *frontend*, l'applicazione mostri

solo quelli disponibili e che, comunque, la coreografia si possa espandere con eccezioni ed altre semplici aggiunte, a costo però di complicarne ulteriormente la leggibilità.

Infine, si noti che, nella modellazione, i task di errore di ACMEMobility sono seguito da *end event* senza il cerchio pieno, per indicare che non tutti i processi del pool devono essere conclusi: per quanto appena visto, altri processi potrebbero essere stati lanciati in parallelo ed essere ancora in esecuzione.

A livello logico, nel BPMN a scopo documentativo il pool di ACMEMobility è costituito da un'unica grande lane, con la sola separazione del task di intervento manuale, relegata alla lane “human assistance”, stando ad indicare una divisione tra i task automatizzati e quelli eseguiti da operatori veri.

Come già accennato, il BPMS di ACME può essere controllato da chiamate *http* — che possiamo simulare con lo script python *user.py* — per avviare o gestire i processi. Al loro arrivo, Camunda ne converte automaticamente i parametri (nel *body* di una *POST*) in variabili globali — dunque disponibili per le implementazioni, per esempio, dei *service task*.

Tra i parametri richiesti dalla “API” di ACME, risultano particolarmente utili *isImmediate* (booleano per distinguere il tipo di noleggio) e, in fase di parcheggio, l'id della stazione (ipoteticamente rilevato dal veicolo stesso tramite comunicazione a corto raggio). Inoltre, la variabile *businessKey*, già implementata e riconosciuta automaticamente in Camunda, permette di associare le richieste a uno stesso processo: usando l'id del veicolo in ogni chiamata, le comunicazioni relative a ogni corsa saranno così aggregate. Dal *cockpit* (GUI) di Camunda, i processi saranno indicati proprio dalla loro *businessKey*.

3) *Prospettiva altri servizi*: Tutti gli altri partecipanti (banca, stazione, FleetManagement, veicolo), nel BPMN documentativo, sono modellati in maniera banale, poiché ogni loro capability viene vista fondamentalmente come un unico *service task*, avviato da una richiesta di ACME e che ritorna un risultato a sua volta. Le uniche eccezioni sono costituite da alcuni loop, per FleetManagement e veicolo, nella fase del **tragitto**. Infatti, quando ACMEMobility le richiede l'inizio del tracking, FleetManagement si mette in ascolto, in attesa di informazioni, fino all'interruzione del tracking; allo stesso tempo, il veicolo noleggiato avrà ricevuto, dallo stesso FleetManagement, una richiesta di iniziare a condividere il suo stato periodicamente (con una *POST* via *http*), anch'esso fino a una notifica di terminazione. Le query di ACME sono invece gestite tramite un task separato di FleetManagement: il motivo è che i dati trasmessi dai veicoli vengono salvati in un database (di FleetManagement) e, al momento dell'interrogazione, vengono ritornati quelli più recenti.

La modellazione raffigura FleetManagement come un unico pool e un'unica lane, invece che dividere i task di condivisione batteria e condivisione posizione, sia per leggibilità (visto che i task sarebbero stati identici), sia perché, dal punto di vista di ACME, esso appare come un unico servizio (implementato con dei microservizi al suo interno).

B. I servizi

I servizi sono stati realizzati tenendo in mente un'architettura in cui ognuno sia agnostico riguardo all'implementazione degli altri, comunicando tramite API documentate (con protocolli *http* e *SOAP*).

Perciò, eventuali dati non-volatili di ogni servizio — e solo quelli necessari — sono memorizzati su un proprio database, accessibile solo da esso e non dall'esterno. Nell'implementazione, essendo più che altro a scopo dimostrativo, le basi di dati sono in realtà solo simulate (tramite file, classi o pacchetti) e accedute tramite opportune interfacce (chiamate di funzione), piuttosto che implementate come veri e propri microservizi a loro volta, interconnessi con gli altri.

Principalmente, (tutti documentati in *docs/implementation/db.md*)

V. RISULTATI

VI. CONCLUSIONE E DISCUSSIONE FINALE

REFERENCES